CLAUDE CERTIFIED ARCHITECT — PROFESSIONAL

Objective 12: Evaluate Tool and Agent Configuration for Capability Bloat

Exam objective: Evaluate tool and agent configurations to identify and remediate capability bloat, including tool selection and distribution, tool definitions and descriptions, redundant tooling, tool grouping and naming, limiting agent capabilities, reducing unnecessary context consumption, and progressive tool discovery.

Q1 **A logistics company's dispatch agent has been given 54 tools spanning route optimization, HR lookups, invoicing, and fleet maintenance so that "any future request can be handled." Users report the agent frequently selects the wrong tool for simple route queries. What is the most likely root cause?**

A The underlying model is too small to handle complex reasoning

B **Tool capability bloat is degrading the model's ability to discriminate between relevant tools for a given task**

C The agent's temperature setting is too high

D The vector database used for retrieval is poorly indexed

CORRECT ANSWER **B**

WHY Overloading an agent with many loosely related tools increases ambiguity in tool selection, since every tool description competes for the model's attention at inference time. Lowering temperature or re-indexing retrieval wouldn't address a tool-selection problem rooted in scope creep, and model size is not the primary driver of this symptom.

Q2 An insurance claims agent has three tools: `get_claim`, `fetch_claim_record`, and `retrieve_claim_details`, each implemented by a different team and returning slightly different schemas. What should the architecture team do first?

A Add a fourth tool that merges the outputs of all three

B Consolidate into a single well-documented `get_claim` tool with a clear schema and deprecate the duplicates

C Increase the context window so the model can use all three without confusion

D Rename all three tools alphabetically so the model picks the first one

CORRECT ANSWER B

WHY Redundant tools with overlapping functionality are a classic source of bloat; consolidating into one authoritative tool with a clear contract reduces ambiguity and context overhead. Merging outputs or expanding the context window treats the symptom rather than the cause, and alphabetical renaming does not reliably influence selection logic.

Q3 A manufacturing operations agent exposes a tool named `do_thing_2` with the description "handles stuff for plant ops." Developers complain the agent rarely invokes it correctly. What is the best remediation?

A Rewrite the tool name and description to be specific and action-oriented, e.g., `get_machine_downtime_report`, with a description of required parameters and expected output

B Remove the tool entirely since it is unused

C Increase the number of few-shot examples in the system prompt until the agent learns the mapping

D Lower the tool's priority ranking in the agent's tool list

CORRECT ANSWER A

WHY Vague naming and descriptions are a primary driver of poor tool selection; clear, specific naming conventions and parameter documentation let the model reason accurately about when to invoke a tool. Removing it discards needed functionality, and few-shot patching or priority tweaks are workarounds rather than fixes to the root metadata problem.

Q4 **A SaaS platform wants to let a single support agent handle billing, account management, and technical troubleshooting, currently modeled as 40 individual tools. Which design change best reduces capability bloat while preserving functionality?**

A **Group related tools into a smaller number of composite, parameterized tools (e.g., `manage_billing(action, params)`) rather than one tool per micro-action**

B Split each tool further into even more granular single-purpose tools

C Disable tool use entirely and rely on the model's training knowledge

D Require the user to manually specify which tool to call in their prompt

CORRECT ANSWER **A**

WHY Grouping related micro-actions into cohesive, parameterized tools reduces the total tool count and context footprint while retaining full capability, improving selection accuracy. Further granularity would worsen bloat, disabling tool use removes needed live-data access, and requiring users to name tools defeats the purpose of an agentic interface.

Q5 **A telecom provider's network-diagnostics agent loads all 80 available diagnostic tools into every conversation, even though a typical session only needs 3-5. What architectural pattern most directly addresses the resulting context consumption?**

A Static system prompt caching only

B **Progressive tool discovery, where the agent first calls a lightweight tool-search/catalog mechanism and loads only relevant tool definitions on demand**

C Increasing the max_tokens parameter to accommodate all tool definitions

D Switching to a larger context-window model so all 80 fit comfortably

CORRECT ANSWER **B**

WHY Progressive discovery lets the agent query a catalog and load only the tools relevant to the current task, directly cutting the number of tool definitions consuming context on every turn. Increasing max_tokens or context window size masks the inefficiency and increases cost/latency without solving the underlying bloat.

Q6 **A government benefits-processing agent has tools for eligibility checks, document verification, and payment issuance, all with identical parameter names (`id`, `data`) across different tools. What risk does this naming pattern introduce?**

A. It increases model confidence in tool selection

B. It increases the chance of parameter confusion and misrouted arguments between structurally similar tools

C. It has no effect since the model infers types automatically

D. It improves caching efficiency

CORRECT ANSWER B

WHY Generic, overlapping parameter naming across tools makes it harder for the model to map arguments correctly, raising the risk of sending the wrong data to the wrong tool. This is a real failure mode, not a caching benefit, and type inference does not resolve ambiguity in which tool should receive which value.

Q7 **An architect reviewing a healthcare agent's configuration finds that 12 of 30 tools have not been invoked in the last 90 days of production logs. What is the recommended action?**

A. Leave them in place in case they are needed later

B. Audit usage data, confirm business relevance, and remove or archive tools that are no longer necessary to shrink the active tool surface

C. Automatically double their invocation priority to encourage usage

D. Merge them into a single fallback "catch-all" tool

CORRECT ANSWER B

WHY Usage-driven audits are the standard way to detect bloat; unused tools should be validated against business need and removed or archived to reduce context cost and selection noise. Keeping unused tools "just in case," boosting their priority, or dumping them into a catch-all tool all increase risk and complexity rather than reducing it.

Q8 A retail chatbot agent is configured with both a `search_products` tool and a broader `general_catalog_query` tool that can also search products among other things. Which grouping principle should guide whether to keep both?

A. Tools should overlap in function to provide redundancy in case one fails

B. Tools should have clearly distinct, non-overlapping responsibilities; if two tools can accomplish the same task, one should be removed or their scopes clarified

C. The agent should always default to the tool listed first in the configuration

D. Overlap is fine as long as descriptions are long enough

CORRECT ANSWER B

WHY Clear separation of responsibilities (single responsibility principle applied to tools) minimizes ambiguity; overlapping tools should be resolved by narrowing scope or removing duplicates. Redundancy-by-overlap is not a standard reliability pattern for tool design and does not compensate for selection ambiguity, nor does verbose description length fix overlapping scope.

Q9 A legal-tech agent used for contract review has 25 tools, most with descriptions exceeding 500 tokens each, consuming a large share of the context window before any user input is processed. What is the most effective fix?

A **Trim descriptions to concise, essential information (purpose, required parameters, constraints) while preserving necessary disambiguation**

B Move all descriptions into the system prompt instead of tool schemas

C Compress descriptions using abbreviations only, without changing content

D Convert all tool descriptions to a different language to save tokens

CORRECT ANSWER **A**

WHY Concise, well-structured tool descriptions reduce the fixed context overhead per conversation while retaining the information the model needs to select and call tools correctly. Moving descriptions to the system prompt doesn't reduce total tokens, abbreviation alone can harm clarity, and language changes don't meaningfully reduce token count for English-trained models.

Q10 An enterprise architecture review finds that an agent's 60 tools are defined with inconsistent naming conventions (`getUser`, `fetch_order`, `RetrieveInvoice-V2`). Besides readability, what is the architectural consequence of this inconsistency?

A It has no measurable effect on model behavior

B Inconsistent naming makes it harder for the model to infer tool families and relationships, increasing misselection risk and complicating future tool-grouping efforts

C It improves the model's ability to generalize across tools

D It reduces the token cost of tool schemas

CORRECT ANSWER B

WHY Consistent naming conventions help the model (and engineers) recognize patterns and groupings among tools; inconsistency increases cognitive and inferential load, raising the chance of incorrect tool choice. It does not reduce token cost, and it actively hurts rather than helps generalization across a tool family.

Q11 **A financial services firm wants a single agent to handle both low-risk FAQ queries and high-risk wire-transfer initiation. From a capability-bloat and risk-control perspective, what is the best design?**

A Give the single agent all tools and rely on prompt instructions to self-restrict risky actions

B **Separate the capabilities into distinct agents or restricted tool subsets, so the FAQ agent cannot access wire-transfer tools at all**

C Add a longer system prompt warning the agent not to misuse the wire-transfer tool

D Keep one agent but increase its temperature for the FAQ path only

CORRECT ANSWER B

WHY Limiting agent capabilities by scoping tool access per use case (least-capability principle) is a stronger control than prompt-based self-restriction, which models can fail to follow under adversarial or ambiguous input. Relying on instructions alone or adjusting temperature does not provide a reliable technical boundary against bloat-driven misuse.

Q12 A hospital's intake assistant agent has tools that return full patient records even when only an appointment confirmation is needed. Which refactor best reduces unnecessary context consumption without reducing functionality?

A Add a separate, narrowly scoped tool (e.g., `get_appointment_status`) that returns only the minimal fields needed, alongside the existing full-record tool for cases that require it

B Keep a single tool but instruct the agent in the prompt to "only look at the fields you need"

C Increase the context window to absorb the larger payloads

D Cache the full patient record tool output indefinitely to avoid repeated calls

CORRECT ANSWER A

WHY Providing purpose-built, minimal-payload tools alongside broader ones lets the agent request only the data it needs for a given task, cutting context consumption and reducing unnecessary PHI exposure. Prompt-based self-restriction is unreliable, expanding the context window increases cost without fixing the inefficiency, and indefinite caching of full records raises its own data-freshness and privacy concerns.

Q13 During a bloat audit, an architect finds an agent configuration where every tool call returns full raw JSON payloads averaging 8,000 tokens, even when only a few fields are relevant to typical queries. What change best controls capability bloat impact at runtime?

A Implement tool output post-processing/filtering so only relevant fields are returned to the model by default, with an option to request full detail when needed

B Ask the model to summarize its own tool output after receiving it, every time

C Reduce the number of tools instead, leaving the payload size unchanged

D Switch to a model with a larger context window to absorb the payload

CORRECT ANSWER A

WHY Filtering tool output server-side to return only relevant fields directly reduces the context consumed per call, which is a payload-level bloat issue distinct from tool-count bloat. Post-hoc model summarization wastes a full round-trip of tokens before trimming, reducing tool count doesn't address oversized responses, and a bigger context window simply absorbs waste rather than eliminating it.

Q14 A multinational bank is designing an agent ecosystem with dozens of specialized sub-agents (fraud detection, KYC, loan servicing). Which practice best prevents future capability bloat as the system scales?

A. **Establish a shared tool-governance process (naming standards, deduplication review, usage audits) applied before new tools are added to any agent**

B. Allow each team to add tools independently without review, trusting engineers to self-regulate

C. Require every new tool to be added to every agent for consistency

D. Freeze the tool catalog permanently once the system launches

CORRECT ANSWER A

WHY A governance process with naming standards and deduplication review is the scalable way to prevent bloat from re-accumulating as more teams contribute tools. Uncontrolled self-regulation tends to reproduce the original problem, pushing every tool into every agent is the definition of bloat, and freezing the catalog prevents legitimate evolution of the system.

Q15 An architect is evaluating whether to split a 45-tool "super-agent" for a logistics platform into multiple specialized agents coordinated by an orchestrator. What is the strongest justification for this split, from a capability-bloat standpoint?

A. It guarantees lower API billing costs regardless of usage patterns

B. Each specialized agent carries only the tools relevant to its domain, improving tool-selection accuracy and reducing per-turn context overhead, while the orchestrator handles routing

C. It eliminates the need for authentication on sub-agents

D. It allows the orchestrator to skip input validation since sub-agents will re-validate

CORRECT ANSWER B

WHY Splitting a monolithic tool set into domain-scoped agents directly reduces the number of tool definitions each agent must reason over, improving selection accuracy and context efficiency, with the orchestrator handling task routing. Cost reduction is not guaranteed and depends on call patterns, and the split has no bearing on authentication requirements or validation responsibilities, which must still be enforced at each boundary. --